

Geração de Malha: Prática

CFD: Curso Básico

Professor: Adriano Possebon Rosa

Departamento de Engenharia Mecânica
Universidade de Brasília

Sumário

- 1 Softwares
- 2 Gmsh
 - Cavidade
 - Cavidade com Malha Estruturada
 - Degrau
 - Tubo
- 3 snappyHexMesh
 - Exemplo.

Alguns softwares utilizados para geração de malha em CFD:

- Ansys (Meshing, ICEM, CFX)
- COMSOL
- Pointwise
- SIMSCALE
- Salome
- cfMesh
- GAMBIT
- STAR-CCM+
- Gmsh
- OpenFOAM (blockMesh, snappyHexMesh)

**Ansys**
part of
synopsys

Products & Services ▾ Learn ▾ About ▾

🔍 🧑 Customers ▾ [Free Trials →](#)

[Home](#) > [Products](#) > [Ansys Meshing](#)

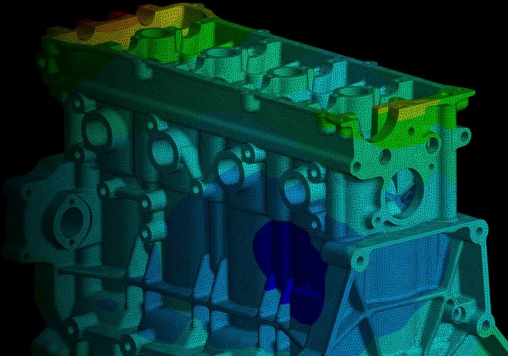
Ansys Meshing

Influencing Accurate Results

Mesh influences the accuracy, convergence and speed of a simulation. Ansys provides tools to produce the most appropriate mesh for accurate, efficient solutions.

[CONTACT US ▾](#)

[Overview](#) [Capabilities](#)



[Who are we ?](#)[Open source software](#)[Services](#)[Resources](#)[FR](#)[EN](#)[Contact us](#)[Simvia](#) › [Open source software](#) › [Salomé](#)

Salomé

Integrated open-source platform for pre- and post-processing

SIEMENS

Brazil | EN

Support & community ▾

Log in

Products & services

Solutions

Industries

Partner ecosystem

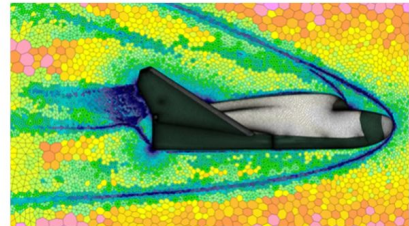
Topics & insights

Search

[Home](#) > Reduce Time to Market by using Simcenter STAR-CCM+ to Automate the CAD-to-Mesh Conversion Process

FACT SHEET

Reduce Time to Market by using Simcenter STAR-CCM+ to Automate the CAD-to-Mesh Conversion Process



Vamos focar em 3 softwares: *blockMesh*, *Gmsh* e *snappyHexMesh*.

Os 3 são *open source*. O *blockMesh* e o *snappyHexMesh* já vêm junto com o OpenFOAM.

O *blockMesh* é interessante para geometrias bem simples.

Para usar o *snappyHexMesh*, precisamos de uma malha de fundo (gerada pelo *blockMesh*) e um arquivo com a geometria de interesse (que pode ser gerado pelo *Gmsh*).

O *Gmsh* possui interface gráfica e é capaz de gerar malhas estruturadas e não estruturadas.

Nossa estratégia vai ser a seguinte.

Para geometrias e malhas bem simples: ***blockMesh***.

Para geometrias razoavelmente complexas: ***Gmsh***.

Para geometrias complexas: ***snappyHexMesh***.

Para malhas adaptativas e malhas móveis: ***snappyHexMesh***.

O *blockMesh* nós já vimos um pouco. Recomendo que você dê uma olhada no *User Guide* e em outros materiais para aprender um pouco mais sobre ele.

Vamos ver o básico do *Gmsh* e do *snappyHexMesh* nos próximos slides.

Comentário antes de começarmos com o *Gmsh*.

Você pode gerar a sua malha com outros softwares e converter para o formato do OpenFOAM.

O próprio OpenFOAM possui diversas *utilities* que possibilitam essa conversão direta: *fluent3DMeshToFoam*, *starToFoam*, *gambitToFoam*, *ideasToFoam*, *cfx4ToFoam*, *gmshToFoam*.

Sumário

- 1 Softwares
- 2 Gmsh
 - Cavidade
 - Cavidade com Malha Estruturada
 - Degrau
 - Tubo
- 3 snappyHexMesh
 - Exemplo.

INTERNATIONAL JOURNAL FOR NUMERICAL METHODS IN ENGINEERING

Int. J. Numer. Meth. Engng 2009; **0**:1–24

Prepared using nmeauth.cls [Version: 2002/09/18 v2.02]

This is a preprint of an article accepted for publication in the International Journal for Numerical Methods in Engineering, Copyright ©2009 John Wiley & Sons, Ltd.

Gmsh: a three-dimensional finite element mesh generator with built-in pre- and post-processing facilities

Christophe Geuzaine¹, Jean-François Remacle^{2*}

¹ *Université de Liège, Department of Electrical Engineering And Computer Science, Montefiore Institute, Liège, Belgium.*

² *Université catholique de Louvain, Institute for Mechanical, Materials and Civil Engineering, Louvain-la-Neuve, Belgium*

SUMMARY

Gmsh is an open-source three-dimensional finite element grid generator with a build-in CAD engine and post-processor. Its design goal is to provide a fast, light and user-friendly meshing tool with parametric input and advanced visualization capabilities. This paper presents the overall philosophy, the main design choices and some of the original algorithms implemented in Gmsh. Copyright © 2009 John Wiley & Sons, Ltd.

KEY WORDS: Computer Aided Design, Mesh generation, Post-Processing, Finite Element Method, Open Source Software

Gmsh é um gerador de malha de elementos finitos open source tridimensional com um CAD incorporado e pós-processador. Seu objetivo de design é fornecer uma ferramenta de geração de malha rápida, leve e fácil de usar, com entrada paramétrica e capacidades flexíveis de visualização.

O Gmsh permite a criação de malhas 1D, 2D e 3D estruturadas (*transfinite*) e não estruturadas. Os elementos podem ser triângulos, quadriláteros, tetraedros, hexaedros, prismas e pirâmides.

O Gmsh pode ser usado de 3 formas:

- por meio da interface gráfica;
- por meio da linguagem específica `.geo` ;
- por meio das linguagens C++, C, Julia e **Python**, através de bibliotecas.

O site do Gmsh é <https://gmsh.info/>. Lá você vai ver como instalar o Gmsh e vai encontrar várias informações sobre o software.

O código fonte pode ser encontrado em <https://gitlab.onelab.info/gmsh/gmsh>.

Uma forma muito interessante de aprender a usar o Gmsh é seguindo os tutoriais oficiais em <https://gmsh.info/doc/texinfo/gmsh.html>

Nos próximos slides vamos ver alguns exemplos simples. Vamos ver como gerar malhas e como usar essas malhas no OpenFOAM.

Sumário

- 1 Softwares
- 2 Gmsh
 - Cavidade
 - Cavidade com Malha Estruturada
 - Degrau
 - Tubo
- 3 snappyHexMesh
 - Exemplo.

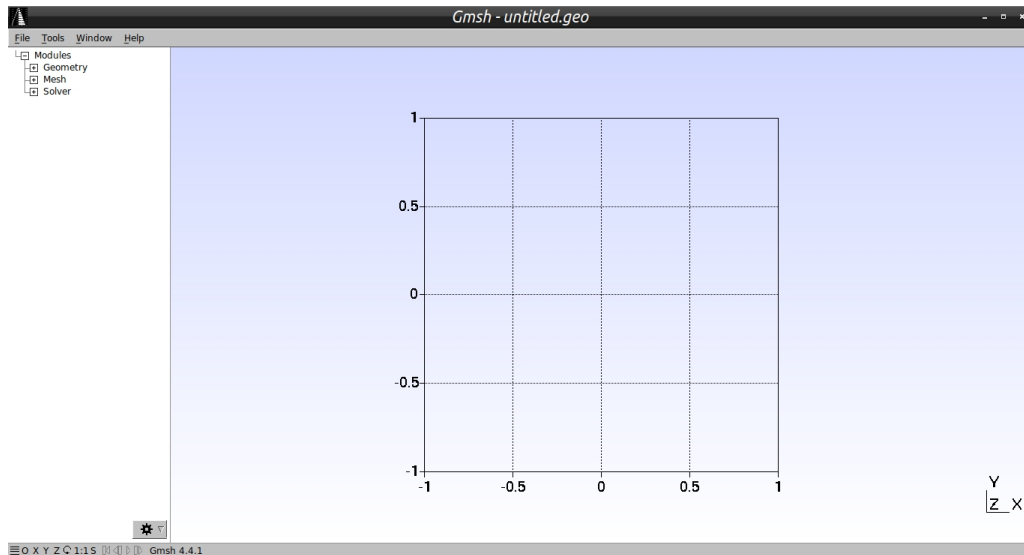
Vamos começar criando a malha para uma cavidade 2D quadrada, de lado 1 m .

Para abrir o Gmsh, dê o comando

```
$ gmsh
```

ou então abra diretamente o software.

Vai aparecer a imagem do próximo slide.

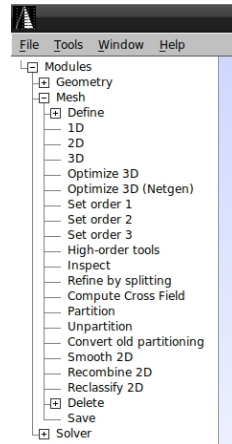
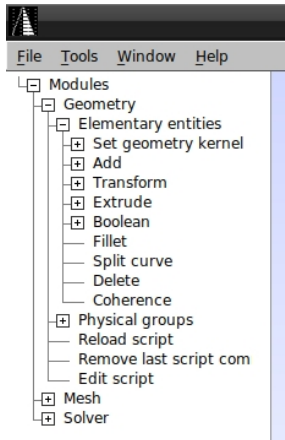


Vá em *File* -> *New ...* e crie um arquivo no formato *geo* . Dê o nome de sua preferência (pode ser *cavity.geo*) .

Escolha a opção *OpenCASCADE* .

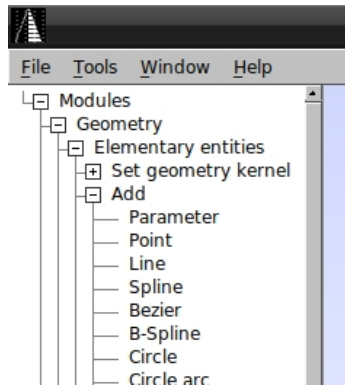
Esse arquivo *.geo* vai salvar tudo que é feito no Gmsh, por meio da interface gráfica.

À esquerda da tela, nós temos os módulos. Vamos usar os módulos *geometry* para gerar a geometria e o *Mesh* para gerar a malha.

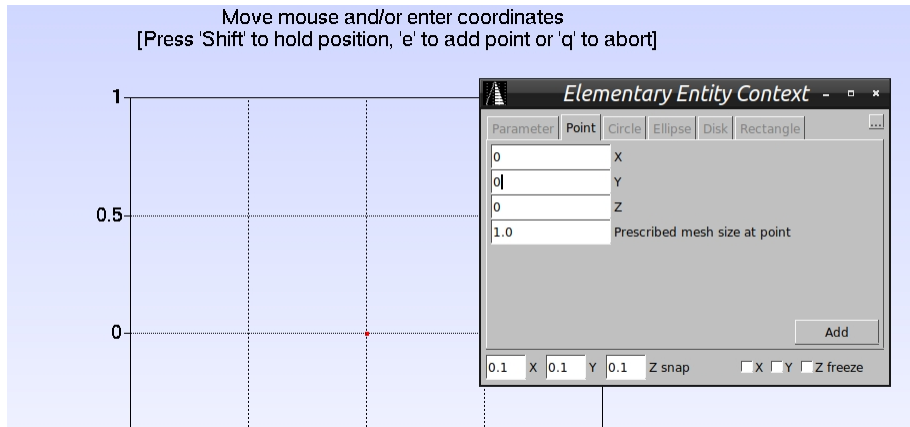


Vamos criar os 4 vértices do quadrado, depois as 4 arestas e por fim a superfície.

Para adicionar um ponto, vá em *Geometry / Elementary entities / Add / Point*.

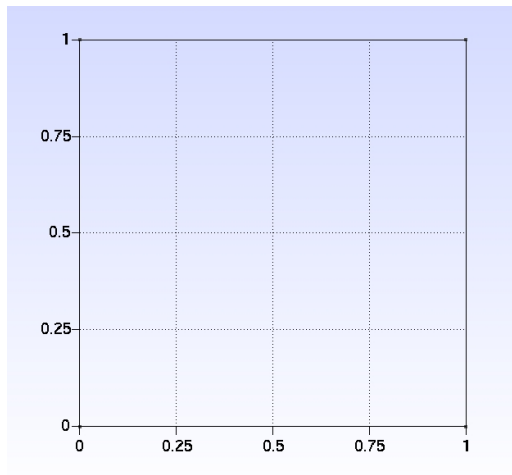


Entre com as coordenadas de cada ponto e clique em *Add*.



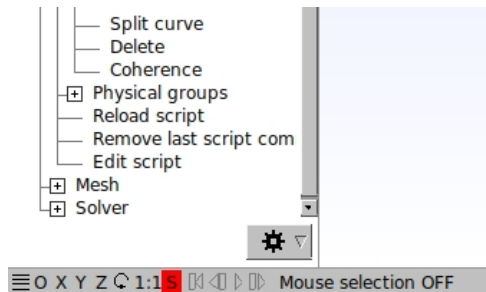
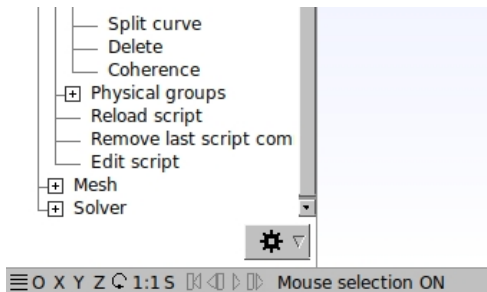
São 4 pontos no total: $(0,0)$, $(1,0)$, $(1,1)$ e $(0,1)$. Deixe a opção “*Prescribed mesh size at point*” como 1.0 mesmo.

Depois de adicionar todos os pontos, feche a janela das coordenadas e digite a letra “q”. Temos os 4 vértices.

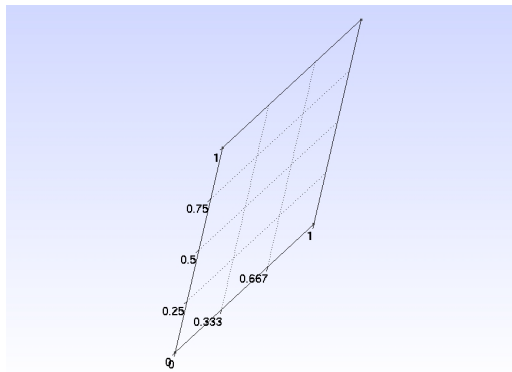


Observação: digite a letra “q” para encerrar um comando. Quando você usa o “Esc”, o que o Gmsh faz é desativar/ativar a seleção com o mouse.

Observe o canto inferior esquerdo da tela.



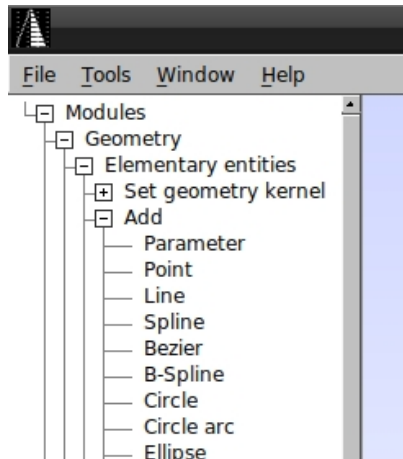
Use os botões do mouse para transladar, girar e dar zoom no desenho.



Para voltar para a visão inicial, clique duas vezes com o botão esquerdo do mouse em algum ponto e selecione ***Reset viewport***.

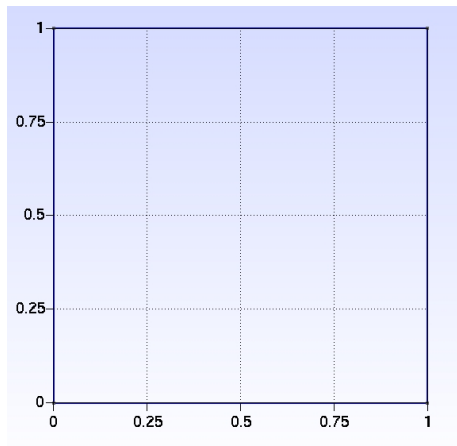
Continuando, vamos criar as arestas (ou linhas).

Clique em *Geometry / Elementary entities / Add / Line* .



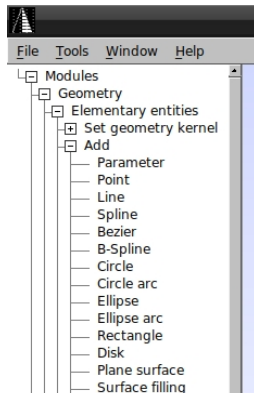
Selecione o ponto inicial e final de cada *line*.

Pressione “q” no final.

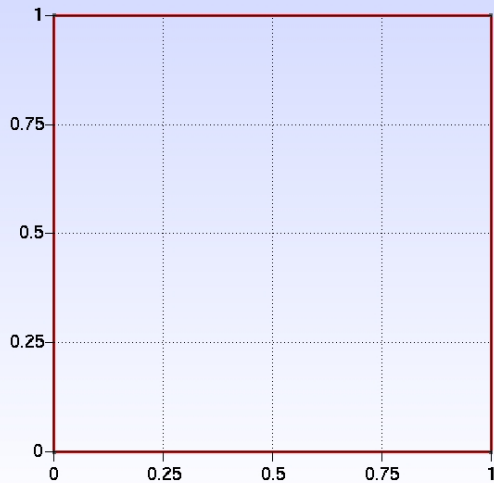


Agora precisamos informar que essas 4 arestas formam uma superfície plana.

Para isso, clique em *geometry/elementary entities/Add/Plane surface*, selecione o quadrado (a linha tracejada no centro do quadrado, para ser mais específico) e digite a letra “e” no teclado.



Select hole boundaries (if none, press 'e')
[Press 'e' to end selection or 'q' to abort]



Passe o mouse sobre os vértices, sobre as arestas e sobre o centro do quadrado. Você vai ver informações sobre essas entidades geométricas.

É um detalhe **muito interessante** sobre o Gmsh: tudo isso que estamos fazendo fica registrado em um arquivo `.geo`.

Esse arquivo fica na pasta onde o Gmsh foi aberto. Esse arquivo contém o código para gerar essa geometria.

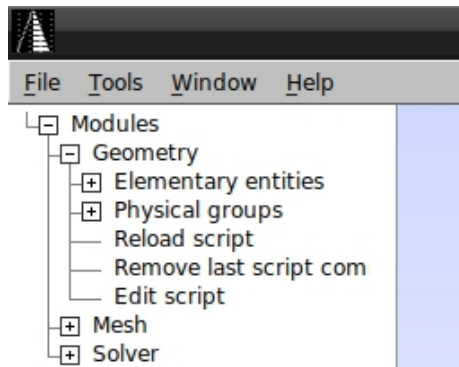
É possível modificar o código, por meio de linhas de texto, e carregar a nova versão no software.

Este é o meu arquivo .geo :

```
1 SetFactory("OpenCASCADE");
2 //+
3 Point(1) = {0, 0, 0, 1.0};
4 //+
5 Point(2) = {1, 0, 0, 1.0};
6 //+
7 Point(3) = {1, 1, 0, 1.0};
8 //+
9 Point(4) = {0, 1, 0, 1.0};
10 //+
11 Line(1) = {1, 2};
12 //+
13 Line(2) = {2, 3};
14 //+
15 Line(3) = {3, 4};
16 //+
17 Line(4) = {4, 1};
18 //+
19 Curve Loop(1) = {4, 1, 2, 3};
20 //+
21 Plane Surface(1) = {1};
```

É possível modificar as posições dos pontos, criar variáveis e incluir trechos de outras geometrias no código.

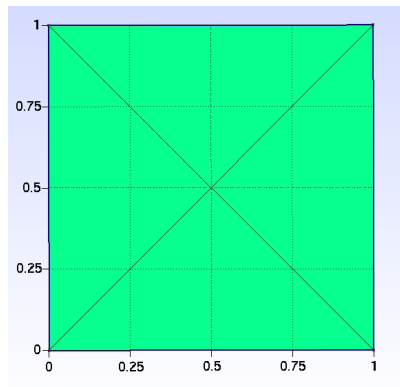
Para abrir o arquivo com o código basta ir em *Geometry / Edit script* . Depois de fazer as modificações clique em *Geometry / Reload script* para atualizar o desenho.



Continuando, com a superfície plana já é possível gerar a malha.

No lado esquerdo da tela, clique em *Mesh / 2D* ,

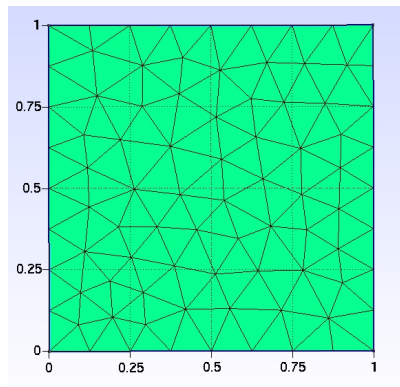
Você deve visualizar algo assim:



(Se você não estiver visualizando a malha, clique duas vezes com o botão esquerdo do mouse e modifique as configurações de *Mesh visibility*, caso 2D.).

Essa malha tem apenas 4 células (na hora de criar os pontos isso foi definido na opção *Prescribed mesh size at point*). Para refinar mais a malha, clique na opção *Mesh / Refine by splitting*.

Cliquei 3 vezes em *Refine by spliting* e depois em *2D* novamente. Este foi o resultado:



Você pode clicar em *Smooth 2D* para o algoritmo gerador de malha tentar deixar as células com um aspecto melhor.

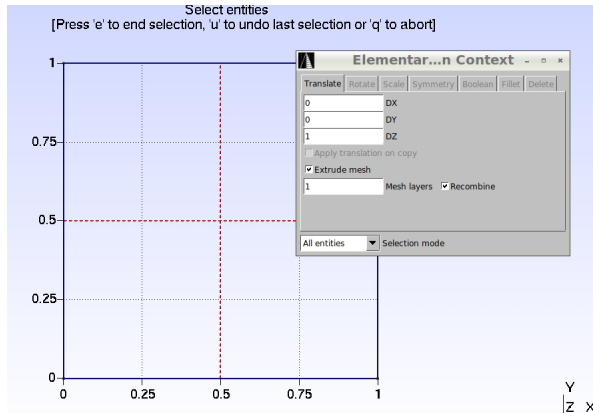
Você também pode clicar em *Recombine* para passar para quadriláteros.

Pronto! Esses são os passos para gerar uma malha 2D no Gmsh: criar uma superfície e depois rodar o comando de malha.

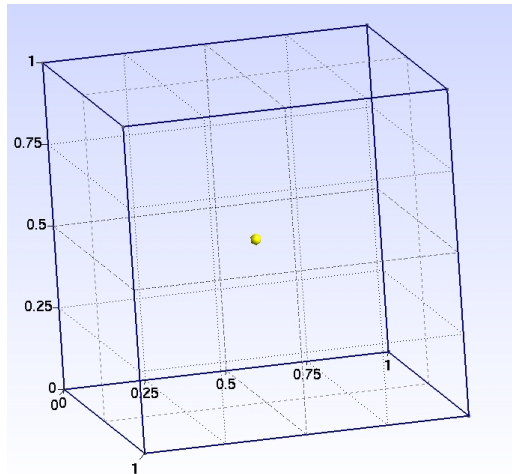
O problema é que o OpenFOAM é 3D. Mesmo para simulações 2D no OpenFOAM, precisamos de uma malha 3D.

Por isso, vamos ter que criar uma geometria 3D a partir de uma extrusão deste quadrado.

Para isso, clique em *Geometry / Extrude / Translate* . Selecione o quadrado, configure a extrusão como está na figura abaixo (extrusão de 1 metro na direção z , com extrusão da malha com apenas uma camada), clique em um ponto qualquer do desenho, digite “e” e depois “q” .



Gire a imagem.



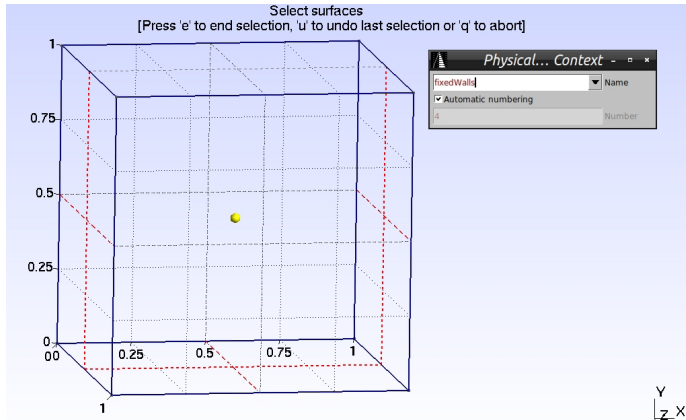
Temos agora uma geometria 3D.

Antes de gerar a malha, vamos definir os nomes das superfícies e dos volumes. Isso vai ser fundamental para o OpenFOAM, para podermos indicar as condições de contorno.

Para isso, vamos usar a opção *Physical groups*. Note que esses nomes que vamos definir não têm nenhum impacto na geometria e nem na malha. Eles servem apenas para que o software reconheça cada superfície.

Clique em *Geometry / Physical groups / Add / Surface* .

Selecione as superfícies da esquerda, da direita e de baixo, dê o nome de *fixedWalls* a elas, clique em qualquer lugar no desenho e digite “e” .

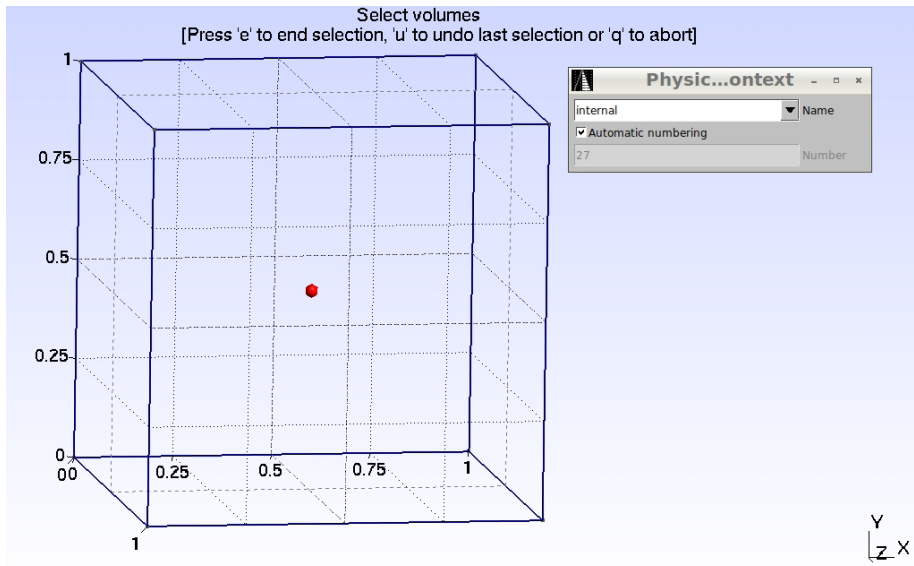


Repita o procedimento para a parede de cima (*movingWall*) e para as paredes de frente e de trás (*frontAndBack*).

Passe o mouse sobre a linha tracejada que passa no centro de cada superfície. Se tudo estiver correto, vai aparecer o nome do *Physical group* que você acabou de definir.

Por fim, clique em *Geometry / Physical groups / Add / Volume*, selecione a esfera amarela (ela vai ficar vermelha), dê o nome de *internal*, clique em qualquer ponto do desenho e digite “e”.

O OpenFOAM vai precisar reconhecer esse volume interno de células.



Acompanhe todas essas modificações no arquivo de texto. Veja como ele foi modificado.

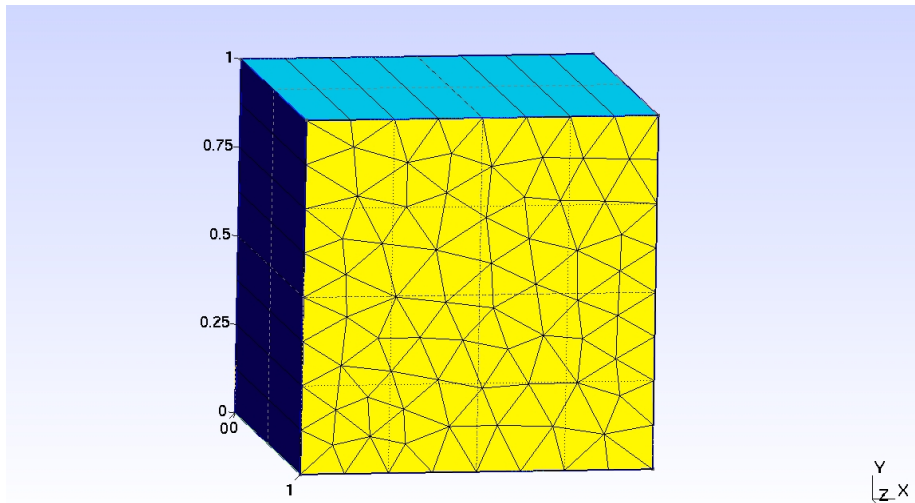
Agora podemos gerar a malha para exportá-la para o OpenFOAM.

Clique em *Mesh/2D* . Clique 3 vezes em *Mesh/Refine by splitting* e depois em *Mesh/2D* de novo .

Por fim, clique em *3D*, já que a malha final tem que ser 3D.

Observe se vai aparecer a mensagem “*Done optimizing 3D mesh*” no canto esquerdo inferior da tela.

Pronto! Agora temos uma malha 3D pronta para o OpenFOAM.



Para exportar malha, vá em *File -> Export*, selecione a opção de formato *Mesh (.msh)*, clique em *Save* e selecione *Version 2 ASCII* na opção de formato.

Abra o arquivo que foi gerado e veja como ele foi configurado.

Terminamos no Gmsh!

No OpenFOAM

Copie o arquivo .msh na pasta principal do caso. Aqui, use o caso da cavidade cisalhante 2D, que vimos na primeira aula (a malha havia sido gerada com o *blockMesh*).

Rode o comando

```
$ gmshToFoam <nome do arquivo>
```

Corrija o *type* de cada *patch* no arquivo *constant/polyMesh/boundary* . Você vai usar os *types wall* e *empty*.

Confira as condições de contorno na pasta 0 e rode o caso.

(Compare o resultado desta malha com a original. Qual é melhor?)

Sumário

- 1 Softwares
- 2 Gmsh
 - Cavidade
 - Cavidade com Malha Estruturada
 - Degrau
 - Tubo
- 3 snappyHexMesh
 - Exemplo.

Para gerar uma cavidade quadrada com malha estruturada, vamos usar o arquivo que já criamos.

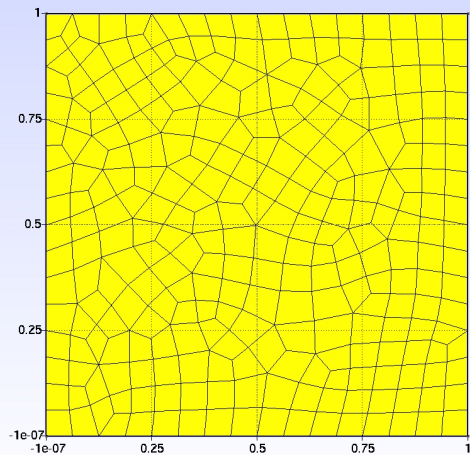
Copie o arquivo .geo e dê outro nome a ele.

Primeiro vamos dizer que queremos quadriláteros e não triângulos. Para isso vamos usar o comando *Recombine*.

Na árvore do lado esquerdo da tela, clique em *Mesh/Recombine*. Selecione a superfície de trás, original (*Plane 1*). Digite “e” e depois “q”.

É importante que seja a superfície *Plane 1* porque ela é a original (as outras surgiram a partir da extrusão).

Apenas por curiosidade, clique em *Mesh/2D* e depois em *Refine by splitting*. Note que agora os elementos são quadriláteros.



Y
Z X

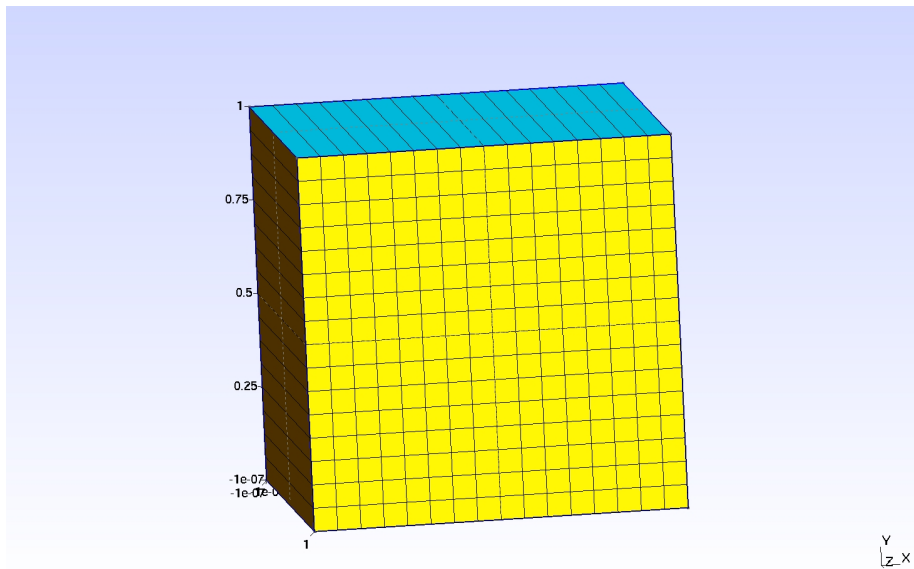
A malha é formada por quadriláteros, mas ainda é não estruturada.

O comando para deixar a malha estruturada é o ***Transfinite***.

Clique em *Mesh/Define/Transfinite/Surface* e selecione a superfície *Plane 1*. Clique em qualquer lugar do desenho e digite “e” e depois “q”.

Clique em *Mesh/2D*, em *Mesh/Refine by splitting* (três vezes) e em *Mesh/2D* de novo.

O resultado está no próximo slide.

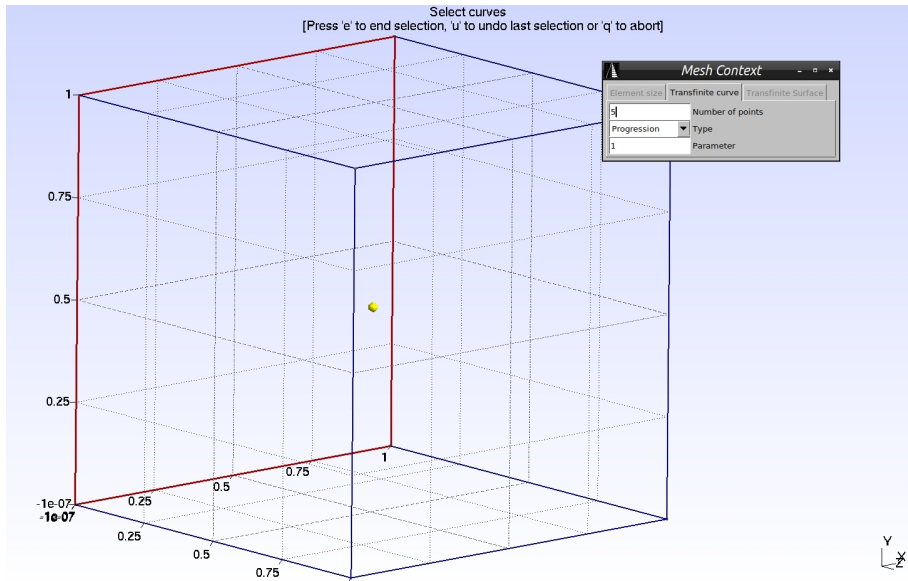


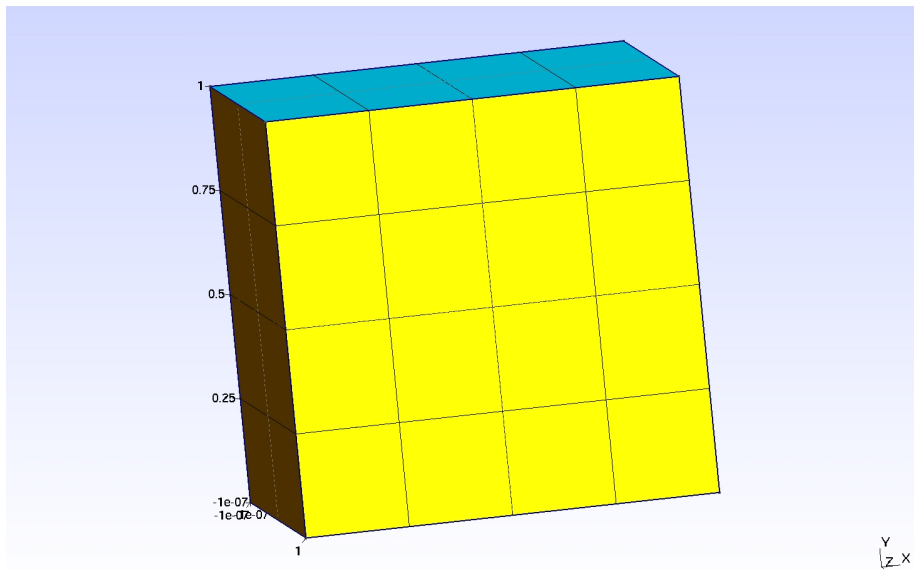
Para ficar ainda melhor, poderíamos definir o número de pontos que queremos em cada aresta. Esse negócio de ficar clicando 3 vezes em *Refine by splitting* é cansativo e nada preciso.

Para isso, clique *Mesh/Define/Transfinite/Curve*, selecione as *Lines* 1, 2, 3 e 4 e defina o número de pontos em cada linha como sendo 5.

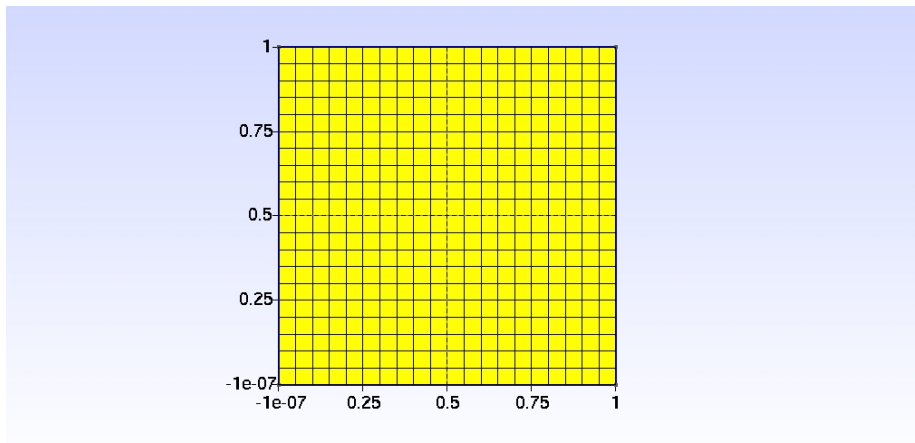
5 pontos correspondem a 4 células em cada linha.

Gere a malha com o comando *Mesh/2D* .





Modifique o arquivo de texto e troque 5 pontos por 21. Entre no Gmsh, clique em *Geometry/Reload script* e depois em *Mesh/2D*.



O código completo ficou assim:

```
1 SetFactory("OpenCASCADE");
2 //+
3 Point(1) = {0, 0, 0, 1.0};
4 //+
5 Point(2) = {1, 0, 0, 1.0};
6 //+
7 Point(3) = {1, 1, 0, 1.0};
8 //+
9 Point(4) = {0, 1, 0, 1.0};
10 //+
11 Line(1) = {1, 2};
12 //+
13 Line(2) = {2, 3};
14 //+
15 Line(3) = {3, 4};
16 //+
17 Line(4) = {4, 1};
```

```
18 //+
19 Curve Loop(1) = {4, 1, 2, 3};
20 //+
21 Plane Surface(1) = {1};
22 //+
23 Extrude {0, 0, 1} {
24   Surface{1}; Layers{1}; Recombine;
25 }
26 //+
27 Physical Surface("fixedWalls") = {2, 3, 4};
28 //+
29 Physical Surface("frontAndBack") = {1, 6};
30 //+
31 Physical Surface("movingWall") = {5};
32 //+
33 Physical Volume("internal") = {1};
34 //+
35 Recombine Surface {1};
36 //+
37 Transfinite Surface {1};
38 //+
39 Transfinite Curve {4, 1, 2, 3} = 10 Using Progression 1;
```

Observações.

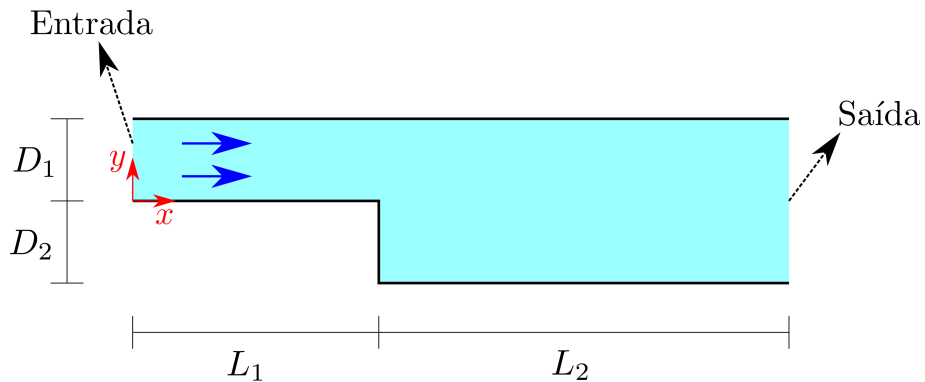
Para ter uma superfície *Transfinite*, ela deve ser um quadrilátero e o número de pontos nos lados opostos deve ser o mesmo.

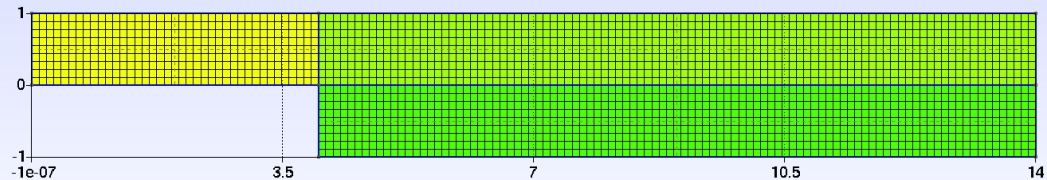
O melhor é usar o *Transfinite/Curve* para uma *Line* de cada vez. Assim fica mais fácil para controlar a progressão (coloque uma progressão de 1.1 e veja o que acontece). Depois, individualize o *Transfinite* e tente novamente.

Para gerar uma malha 3D com várias divisões ao longo da direção z , basta alterar o valor de *Layers* no momento da extrusão (procure, no arquivo geo, o comando de extrusão e altere o valor de 1 para 10; depois dê um *Reload* no Gmsh e gere a malha novamente).

Sumário

- 1 Softwares
- 2 Gmsh
 - Cavidade
 - Cavidade com Malha Estruturada
 - Degrau
 - Tubo
- 3 snappyHexMesh
 - Exemplo.





Y
Z X

O código completo ficou assim:

```
1  SetFactory("OpenCASCADE");
2
3  D1 = 1.0;
4  D2 = D1;
5  L1 = 4*D1;
6  L2 = 10*D1;
7
8  N_D1 = 10;
9  N_D2 = (D2/D1)*N_D1;
10 N_L1 = (L1/D1)*N_D1;
11 N_L2 = (L2/D1)*N_D1;
12
13 //+
14 Point(1) = {0, 0, 0, 1.0};
15 //+
16 Point(2) = {L1, 0, 0, 1.0};
17 //+
18 Point(3) = {L1, -D2, 0, 1.0};
19 //+
20 Point(4) = {L1+L2, -D2, 0, 1.0};
```

```
21 //+
22 Point(5) = {L1+L2, 0, 0, 1.0};
23 //+
24 Point(6) = {L1+L2, D1, 0, 1.0};
25 //+
26 Point(7) = {L1, D1, 0, 1.0};
27 //+
28 Point(8) = {0, D1, 0, 1.0};
29 //+
30 Line(1) = {1, 2};
31 //+
32 Line(2) = {3, 2};
33 //+
34 Line(3) = {3, 4};
35 //+
36 Line(4) = {4, 5};
37 //+
38 Line(5) = {2, 5};
39 //+
40 Line(6) = {5, 6};
41 //+
42 Line(7) = {1, 8};
```

```
43 //+
44 Line(8) = {8, 7};
45 //+
46 Line(9) = {7, 6};
47 //+
48 Line(10) = {2, 7};
49 //+
50 Curve Loop(1) = {8, -10, -1, 7};
51 //+
52 Plane Surface(1) = {1};
53 //+
54 Curve Loop(2) = {5, 6, -9, -10};
55 //+
56 Plane Surface(2) = {2};
57 //+
58 Curve Loop(3) = {2, 5, -4, -3};
59 //+
60 Plane Surface(3) = {3};
61 //+
62 Recombine Surface {1, 2, 3};
63 //+
64 Transfinite Curve {7, 10, 6} = N_D1 Using Progression 1;
```

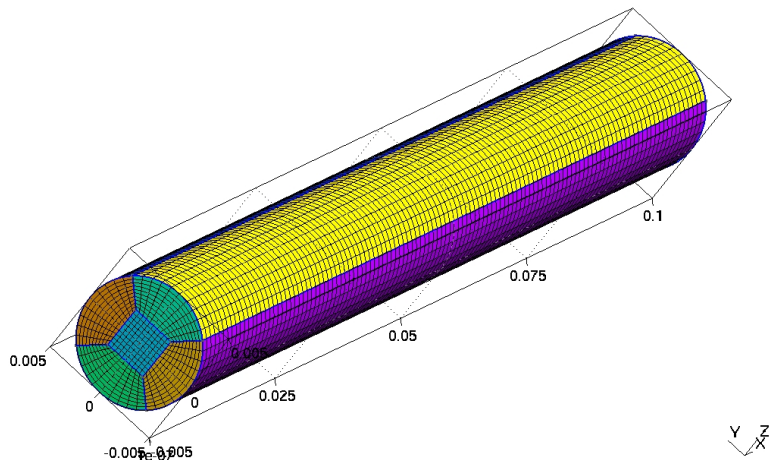
```
65 //+
66 Transfinite Curve {8, 1} = N_L1 Using Progression 1;
67 //+
68 Transfinite Curve {9, 5, 3} = N_L2 Using Progression 1;
69 //+
70 Transfinite Curve {2, 4} = N_D2 Using Progression 1;
71 //+
72 Transfinite Surface {1};
73 //+
74 Transfinite Surface {2};
75 //+
76 Transfinite Surface {3};
77 //+
78 Extrude {0, 0, 1} {
79     Surface{1}; Surface{2}; Surface{3}; Layers{1}; Recombine;
80 }
81 //+
82 Physical Surface("inlet") = {7};
83 //+
84 Physical Surface("outlet") = {10, 14};
85 //+
86 Physical Surface("frontAndBack") = {8, 12, 16, 3, 2, 1};
```

```
87 //+  
88 Physical Surface("fixedWalls") = {11, 4, 6, 13, 15};  
89 //+  
90 Physical Volume("internal") = {1, 2, 3};
```

Sumário

- 1 Softwares
- 2 Gmsh
 - Cavidade
 - Cavidade com Malha Estruturada
 - Degrau
 - **Tubo**
- 3 snappyHexMesh
 - Exemplo.

Tubo com 1 *cm* de diâmetro e 10 *cm* de comprimento.



O código completo ficou assim:

```
1 SetFactory("OpenCASCADE");
2 //+
3 d = 0.01;
4 //+
5 L = 0.1;
6 //+
7 cos = 0.70710678;
8 //+
9 Point(1) = {0, 0, 0, 1.0};
10 //+
11 Point(2) = {0, cos*d/4, cos*d/4, 1.0};
12 Point(3) = {0, -cos*d/4, cos*d/4, 1.0};
13 Point(4) = {0, -cos*d/4, -cos*d/4, 1.0};
14 Point(5) = {0, cos*d/4, -cos*d/4, 1.0};
15 //+
16 Point(6) = {0, d*cos/2, d*cos/2, 1.0};
17 Point(7) = {0, -d*cos/2, d*cos/2, 1.0};
18 Point(8) = {0, -d*cos/2, -d*cos/2, 1.0};
19 Point(9) = {0, d*cos/2, -d*cos/2, 1.0};
```

```
20 //+
21 Circle(1) = {9, 1, 6};
22 Circle(2) = {6, 1, 7};
23 Circle(3) = {7, 1, 8};
24 Circle(4) = {8, 1, 9};
25 //+
26 Line(5) = {5, 2};
27 Line(6) = {2, 3};
28 Line(7) = {3, 4};
29 Line(8) = {4, 5};
30 Line(9) = {8, 4};
31 Line(10) = {9, 5};
32 Line(11) = {6, 2};
33 Line(12) = {7, 3};
34 //+
35 Curve Loop(1) = {4, 10, -8, -9};
36 Plane Surface(1) = {1};
37 Curve Loop(2) = {1, 11, -5, -10};
38 Plane Surface(2) = {2};
```

```
39 Curve Loop(3) = {2, 12, -6, -11};
40 Plane Surface(3) = {3};
41 Curve Loop(4) = {3, 9, -7, -12};
42 Plane Surface(4) = {4};
43 Curve Loop(5) = {8, 5, 6, 7};
44 Plane Surface(5) = {5};
45 //+
46 Recombine Surface {1, 2, 3, 4, 5};
47 //+
48 Transfinite Curve {9, 10, 11, 12} = 11 Using Progression 1;
49 Transfinite Curve {4, 1, 2, 3, 8, 7, 6, 5} = 11 Using Progression 1;
50 Transfinite Surface {4};
51 Transfinite Surface {1};
52 Transfinite Surface {2};
53 Transfinite Surface {3};
54 Transfinite Surface {5};
55 //+
56 Extrude {L, 0, 0} {
57   Surface{3}; Surface{2}; Surface{1}; Surface{4}; Surface{5}; Layers{120}; Recombine;
58 }
```

```
59 //+
60 Physical Surface("inlet") = {3, 2, 1, 4, 5};
61 Physical Surface("wall") = {19, 8, 13, 15};
62 Physical Surface("outlet") = {14, 18, 21, 10, 22};
63 Physical Volume("internal") = {2, 3, 5, 1, 4};
```

Sumário

- 1 Softwares
- 2 Gmsh
 - Cavidade
 - Cavidade com Malha Estruturada
 - Degrau
 - Tubo
- 3 snappyHexMesh
 - Exemplo.

Esta seção descreve a *utility* para geração de malha, ***snappyHexMesh***, que já vem com o OpenFOAM.

O ***snappyHexMesh*** *utility* gera malhas tridimensionais contendo hexaedros (*hex*) e hexaedros divididos (*split-hex*) automaticamente a partir de geometrias de superfícies trianguladas, ou *tri-surfaces*.

Para rodar o ***snappyHexMesh*** é preciso ter uma geometria no formato STL (Stereolithography) ou OBJ (Wavefront Object).

Gerar a malha com o *snappyHexMesh* é uma simulação por si só e pode demorar horas, dependendo do número de células e complexidade da malha.

São feitas várias iterações até que a malha esteja envolvendo a geometria de entrada, de acordo com os parâmetros de entrada.

Uma fase opcional encolherá a malha resultante e inserirá camadas de células em volta das superfícies.

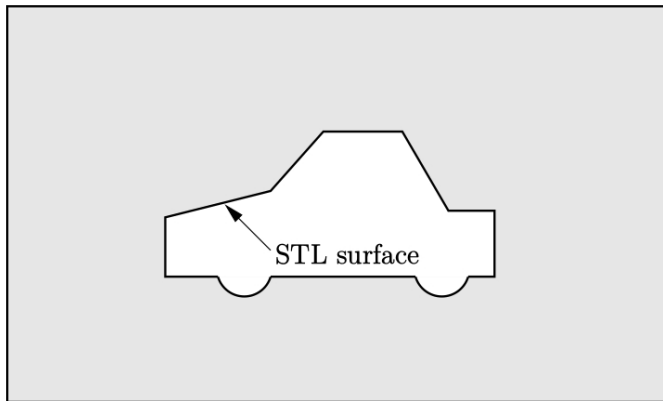
A especificação do nível de refinamento da malha é muito flexível e o tratamento da superfície é robusto, com uma qualidade de malha final pré-especificada.

Para aprender a usar o *snappy*, recomendo

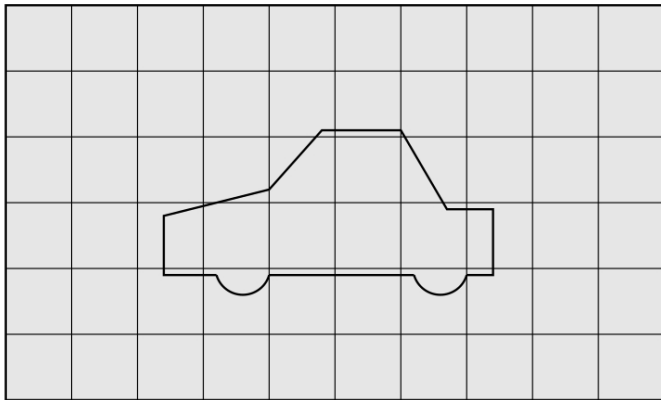
- o *User Guide* oficial;
- os slides da *Wolf Dynamics* (São excelentes!! Vou disponibilizar pra vocês.);
- os tutorias do OpenFOAM (exemplos: *incompressibleFluid/wingMotion*, *incompressibleFluid/motorBike* e *multicomponentFluid/membrane*).

O que o *snappyHexMesh* faz?

Ele pega a geometria que você criou (stl ou obj).



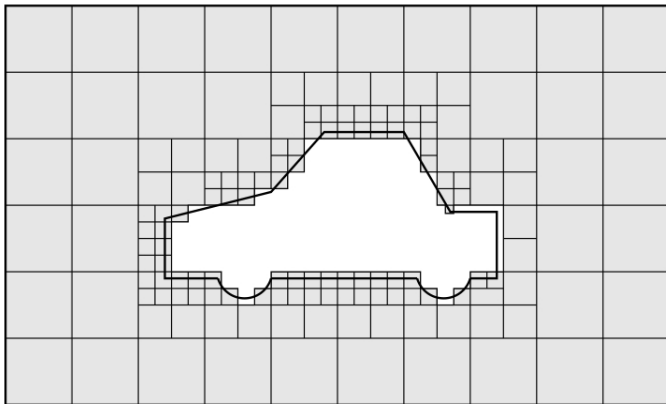
E uma malha de fundo, formada por hexaedros.



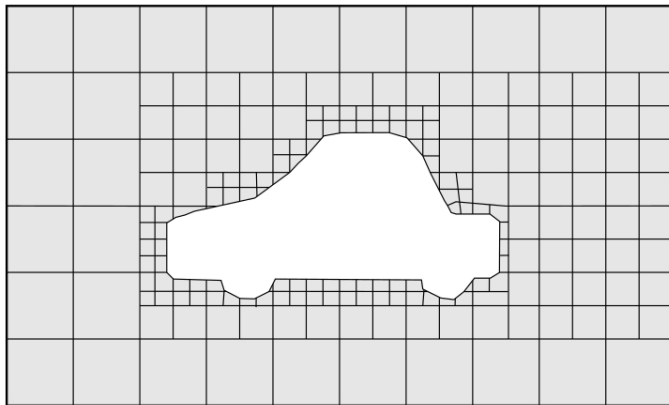
O primeiro passo é identificar as células que estão em contato com a superfície.



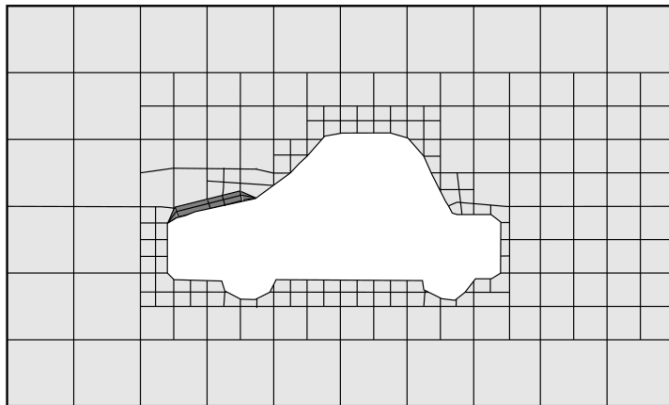
E depois remover essas células são removidas. Esse passo é o ***castellate***.



O penúltimo passo é chamado de ***snap*** e consiste em grudar as células na superfície, adequando melhor a malha à geometria.



O último passo, que na verdade é opcional, consiste em adicionar camadas de células às superfícies. Esse passo é chamado de ***addLayers***.



Em resumo, temos 3 passos:

- *castellate*
- *snap*
- *addLayers*

Esses 3 passos possuem diversos parâmetros de configuração.

Para gerar uma malha com o snappyHexMesh, vamos precisar de:

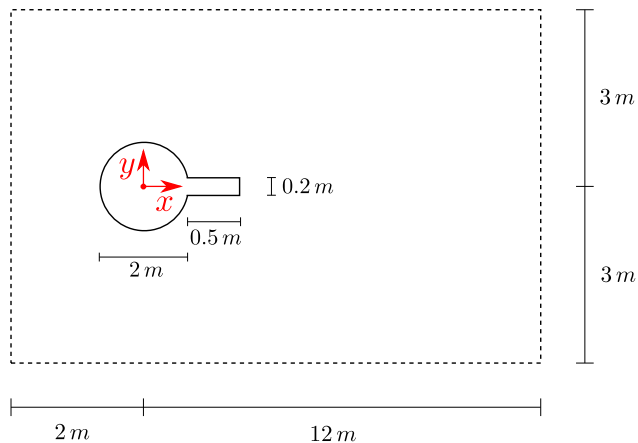
- 1 uma malha de fundo formada por hexaedros (vamos usar o *blockMesh* para gerar essa malha);
- 2 um arquivo *stl* ou *obj* com a geometria em volta da qual ou dentro da qual queremos gerar a malha (podemos usar o *Gmsh* para gerar essa geometria, ou qualquer outro software);
- 3 um arquivo *snappyHexMeshDict* configurado.

Nos próximos slides vamos ver um exemplo.

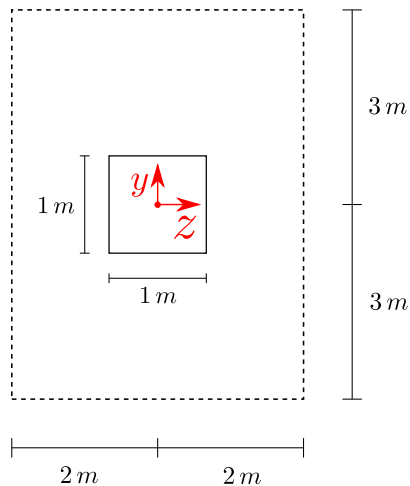
Sumário

- 1 Softwares
- 2 Gmsh
 - Cavidade
 - Cavidade com Malha Estruturada
 - Degrau
 - Tubo
- 3 snappyHexMesh
 - Exemplo.

Vamos gerar a malha em torno de um cilindro circular com uma estrutura acoplada a jusante, como mostra a figura. (Esse é um ótimo tema para o trabalho final.)



Essa estrutura é chamada de *cylinder with splitter plate*. Um objetivo é diminuir/modificar a intensidade da região de esteira.



Antes de começarmos, crie uma pasta, que vai ser o nosso caso de estudo.

Copie um arquivo *controlDict* para uma pasta *system*, dentro da pasta principal do caso.

Sem o arquivo *system/controlDict* em um caso, não é possível rodar nada.

Vamos começar com a malha de fundo. Para gerar essa malha, vamos usar o *blockMesh*.

É importante que as células tenham razão de aspecto próximo de 1.

Também é importante ter células que tenham interseção com a geometria.

Observação: como o nosso problema é $3D$, vou usar uma malha pouco refinada para poder ter uma simulação rápida. (Para melhorar essa malha, você poderia colocar mais ponto no *blockMeshDict*.)

O arquivo *blockMeshDict* ficou assim:

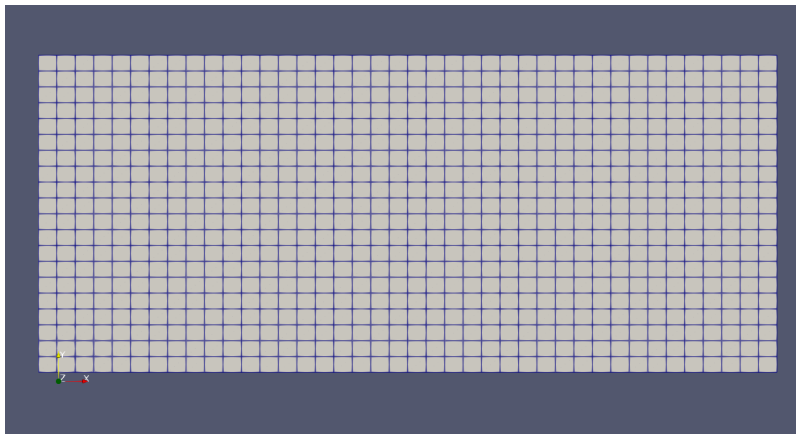
```
1  convertToMeters 1;
2  vertices
3  (
4      (-2 -3 -2)      //0
5      (12 -3 -2)      //1
6      (12 3 -2)       //2
7      (-2 3 -2)       //3
8      (-2 -3 2)       //4
9      (12 -3 2)       //5
10     (12 3 2)        //6
11     (-2 3 2)        //7
12 );
13 blocks
14 (
15     hex (0 1 2 3 4 5 6 7) (40 20 12) simpleGrading (1 1 1)
16 );
17 edges
18 (
19 );
```

```
20 boundary
21 (
22     frontAndBack
23     {
24         type wall;
25         faces
26         (
27             (0 3 2 1)
28             (4 5 6 7)
29         );
30     }
31     inlet
32     {
33         type patch;
34         faces
35         (
36             (0 4 7 3)
37         );
38     }
```

```
39     outlet
40     {
41         type patch;
42         faces
43         (
44             (2 6 5 1)
45         );
46     }
47     bottom
48     {
49         type wall;
50         faces
51         (
52             (1 5 4 0)
53         );
54     }
```

```
55     top
56     {
57         type wall;
58         faces
59         (
60             (3 7 6 2)
61         );
62     }
63 );
```

Agora é só rodar o comando *blockMesh* para gerarmos a nossa malha de fundo.



Agora vamos gerar a nossa geometria, que é o arquivo *stl*.

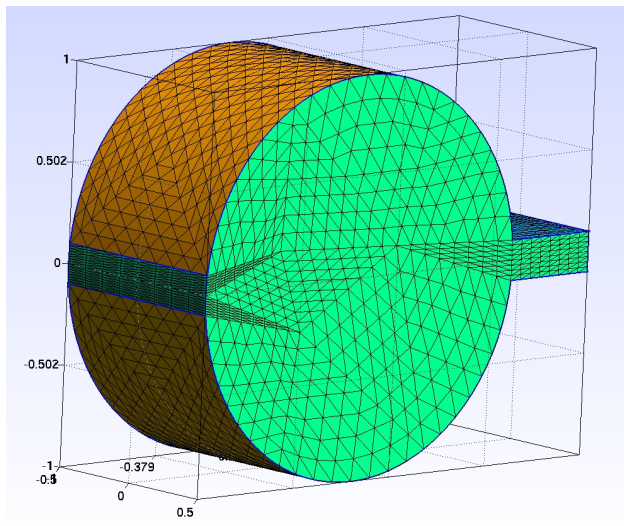
Para isso vamos usar o *Gmsh*.

O código utilizado gerar a geometria está nos próximos slides.

Arquivo *cylinder.geo*:

```
1  SetFactory("OpenCASCADE");
2  //+
3  Point(1) = {0, 0, 0.5, 1.0};
4  Point(2) = {1, 0.1, 0.5, 1.0};
5  Point(3) = {1, -0.1, 0.5, 1.0};
6  Point(4) = {-1, 0.1, 0.5, 1.0};
7  Circle(1) = {2, 1, 4};
8  Point(5) = {-1, -0.1, 0.5, 1.0};
9  Circle(2) = {4, 1, 5};
10 Circle(3) = {5, 1, 3};
11 Point(6) = {1.5, 0.1, 0.5, 1.0};
12 Point(7) = {1.5, -0.1, 0.5, 1.0};
13 Line(4) = {2, 6};
14 Line(5) = {6, 7};
15 Line(6) = {7, 3};
```

```
16 //+
17 Curve Loop(1) = {1, 2, 3, -6, -5, -4};
18 //+
19 Plane Surface(1) = {1};
20 //+
21 Extrude {0, 0, -1} {
22     Surface{1};
23 }
24 //+
25 Physical Volume("internal") = {1};
26 //+
27 Physical Surface("cylinder") = {4, 8, 3, 2, 7, 5, 6, 1};
28 //+
29 Mesh.Algorithm = 6;
30 //+
31 Mesh 6;
32 //+
33 RefineMesh;
34 RefineMesh;
35 RefineMesh;
```



Esse *RefineMesh* no final é importante para gerar um arquivo *stl* de boa qualidade, o que garante um melhor detalhamento da geometria.

No *Gmsh*, exporte a geometria como *stl*.

Salve no formato *ASCII* e com a opção *Per physical surface*.

Coloque o arquivo *cylinder.stl* em *constant/geometry*.

Agora vamos para o arquivo principal, que é o *snappyHexMeshDict*.

As principais entradas no arquivo são:

- castellatedMesh
- snap
- addLayers
- geometry
- castellatedMeshControls
- snapControls
- addLayersControls
- meshQualityControls

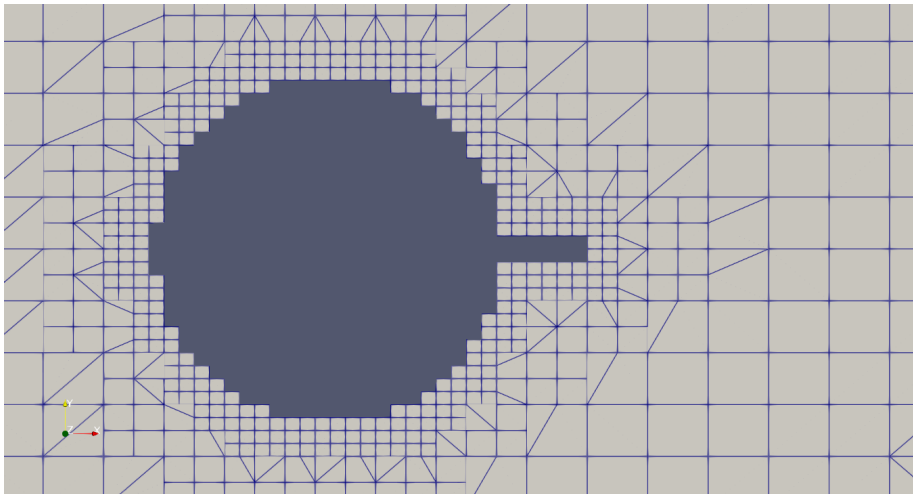
Vamos ver o arquivo completo e uma explicação sobre cada entrada.

Arquivo *snappyHexMeshDict*.

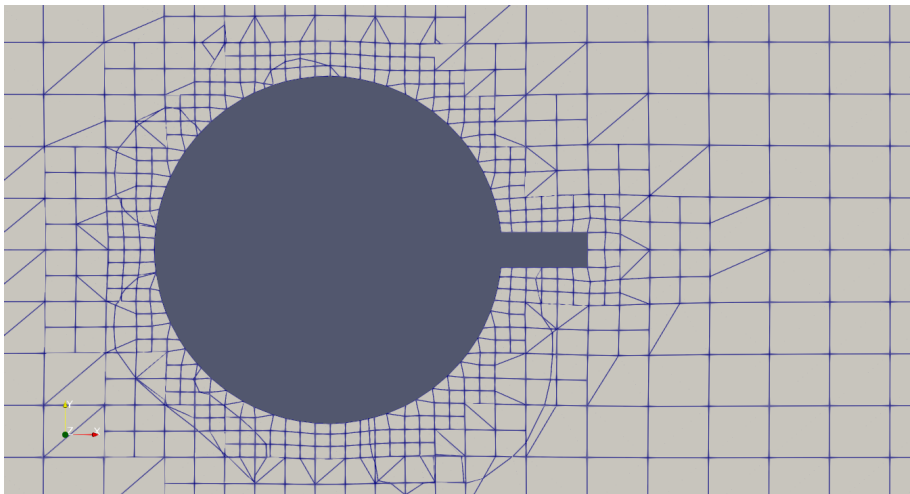
```
1 FoamFile
2 {
3     format      ascii;
4     class       dictionary;
5     object      snappyHexMeshDict;
6 }
7 // * * * * *
8
9 // Which of the steps to run
10 castellatedMesh true;
11 snap           true;
12 addLayers      true;
```

Primeiro nós escolhemos quais dos passos rodar. Podemos rodar o *castellatedMesh* apenas. Ou então rodar o *castellatedMesh* uma vez e depois modificar o *snap* e o *addLayers* e ficar rodando apenas os dois, sem precisar rodar o *castellatedMesh* toda vez.

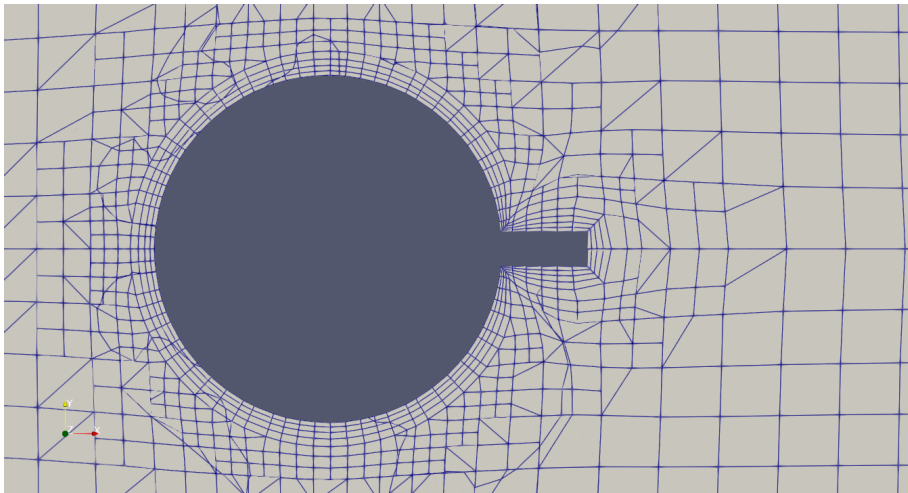
O *castellatedMesh* faz isto:



O *snap* faz isto:



O *addLayers* faz isto:



Vamos deixar os 3 *true*. Continuando no código, vamos para a geometria.

```
13 // Geometry. Definition of all surfaces. All surfaces are of class
14 // searchableSurface.
15 // Surfaces are used
16 // - to specify refinement for any mesh cell intersecting it
17 // - to specify refinement for any mesh cell inside/outside/near
18 // - to 'snap' the mesh boundary to the surface
19 geometry
20 {
21     cylinder
22     {
23         type triSurfaceMesh;
24         file "cylinder.stl";
25     }
26     refinementBox
27     {
28         type searchableBox;
29         min (4.0 -2.0 -0.5);
30         max (6.0 2.0 0.5);
31     }
32 };
```

Note que no próprio arquivo temos várias explicações sobre os parâmetros. Esse arquivo eu peguei do tutorial *incompressibleFluid/motorBike*.

É no dicionário *geometry* que nós informamos qual é o arquivo stl que vamos usar.

Note que eu criei uma região, chamada *refinementBox*, onde a malha será refinada. Isso é opcional. Coloquei aqui para mostrar essa opção, que é muito útil.

Informada a geometria, vamos para o *castellatedMeshControls*, que armazena os parâmetros de controle do *castellatedMesh*.

Arquivo *snappyHexMeshDict*.

```
33 // Settings for the castellatedMesh generation.
34 castellatedMeshControls
35 {
36     // Refinement parameters
37     // ~~~~~
38
39     // If local number of cells is >= maxLocalCells on any processor
40     // switches from refinement followed by balancing
41     // (current method) to (weighted) balancing before refinement.
42     maxLocalCells 100000;
43
44     // Overall cell limit (approximately). Refinement will stop immediately
45     // upon reaching this number so a refinement level might not complete.
46     // Note that this is the number of cells before removing the part which
47     // is not 'visible' from the keepPoint. The final number of cells might
48     // actually be a lot less.
49     maxGlobalCells 1000000;
```

O parâmetro *maxLocalCells* é utilizado para definir o número máximo de células que um processo paralelo pode ter no contexto do refinamento de malha local.

Durante a execução do snappyHexMesh, a malha é refinada localmente em certas regiões da geometria onde é necessário um maior detalhamento para a simulação CFD.

Já o parâmetro *maxGlobalCells* é utilizado para definir o número máximo total de células que a malha pode ter após o refinamento global e local durante o processo de geração de malha.

Ele controla o número máximo de células no domínio global da malha, incluindo todas as regiões refinadas localmente.

```
50 // The surface refinement loop might spend lots of iterations refining just a
51 // few cells. This setting will cause refinement to stop if <= minimumRefine
52 // are selected for refinement. Note: it will at least do one iteration
53 // (unless the number of cells to refine is 0)
54 minRefinementCells 0;
55
56 // Allow a certain level of imbalance during refining
57 // (since balancing is quite expensive)
58 // Expressed as fraction of perfect balance (= overall number of cells /
59 // nProcs). 0=balance always.
60 maxLoadUnbalance 0.10;
61
62 // Number of buffer layers between different levels.
63 // 1 means normal 2:1 refinement restriction, larger means slower
64 // refinement.
65 nCellsBetweenLevels 3;
```

```
66 // Explicit feature edge refinement
67 // ~~~~~
68
69 // Specifies a level for any cell intersected by its edges.
70 // This is a featureEdgeMesh, read from constant/geometry for now.
71 features
72 (
73     {
74         file "cylinder.eMesh";
75         level 1;
76     }
77 );
78
79 // Surface based refinement
80 // ~~~~~
81
82 // Specifies two levels for every surface. The first is the minimum level,
83 // every cell intersecting a surface gets refined up to the minimum level.
84 // The second level is the maximum level. Cells that 'see' multiple
85 // intersections where the intersections make an
86 // angle > resolveFeatureAngle get refined up to the maximum level.
```

Esse parâmetro *features* é opcional. Ele está relacionado com a *utility surfaceFeatures*.

A opção *features* é usada para especificar as características da superfície do objeto geométrico para melhorar a qualidade da malha. O parâmetro *level* dentro da opção *features* controla o nível de detalhe (refinamento) aplicado às características da superfície.

O *level* indica quantas vezes as características da superfície, como bordas afiadas ou cantos, serão refinadas. Quanto maior o valor de *level*, mais refinada será a malha nessas características específicas.

Se o *features* for utilizado, é necessário dar o comando *surfaceFeatures* antes do comando *snappyHexMesh*.

Se o *features* não for utilizado, basta deixar *features()* ; .

```
87 refinementSurfaces
88 {
89     cylinder
90     {
91         // Surface-wise min and max refinement level
92         level (2 2);
93
94         // Optional specification of patch type (default is wall). No
95         // constraint types (cyclic, symmetry) etc. are allowed.
96         patchInfo
97         {
98             type wall;
99         }
100     }
101 }
102
103 // Resolve sharp angles
104 resolveFeatureAngle 30;
```

O *level* determina quantas vezes a malha é refinada nas superfícies especificadas.

Quanto maior o valor de *level*, mais refinada será a malha nesses locais.

O *type* é definido no dicionário *patchInfo*. O padrão é *wall*.

```
105 // Region-wise refinement
106 // ~~~~~
107
108 // Specifies refinement level for cells in relation to a surface. One of
109 // three modes
110 // - distance. 'levels' specifies per distance to the surface the
111 //   wanted refinement level. The distances need to be specified in
112 //   descending order.
113 // - inside. 'levels' is only one entry and only the level is used. All
114 //   cells inside the surface get refined up to the level. The surface
115 //   needs to be closed for this to be possible.
116 // - outside. Same but cells outside.
117
118 refinementRegions
119 {
120     refinementBox
121     {
122         mode    inside;
123         level   1;
124     }
125 }
```

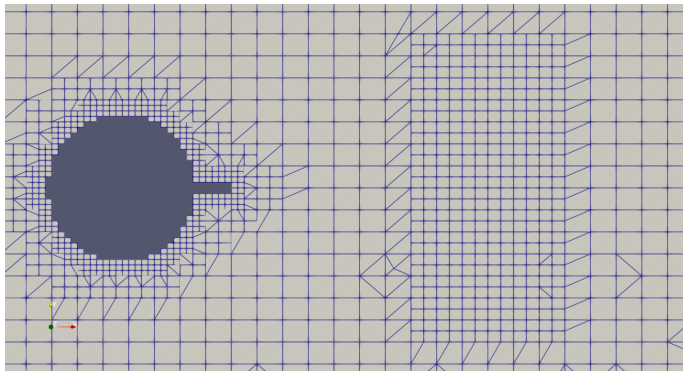
```
126 // Mesh selection
127 // ~~~~~
128
129 // After refinement patches get added for all refinementSurfaces and
130 // all cells intersecting the surfaces get put into these patches. The
131 // section reachable from the locationInMesh is kept.
132 // NOTE: This point should never be on a face, always inside a cell, even
133 // after refinement.
134 locationInMesh (0.0001 1.5001 0);
135
136 // Whether any faceZones (as specified in the refinementSurfaces)
137 // are only on the boundary of corresponding cellZones or also allow
138 // free-standing zone faces. Not used if there are no faceZones.
139 allowFreeStandingZoneFaces true;
140 }
```

Atenção: o parâmetro *locationInMesh* é muito importante.

Este ponto define se a malha será gerada dentro ou fora da geometria informada.

É importante garantir que este ponto não esteja em uma face.

Este é o resultado do *castellatedMesh*. Note a região com um refinamento maior a jusante do cilindro. Essa região é a *refinementBox* que nós criamos.



Agora vamos configurar os parâmetros da fase de *snap*.

```
141 // Settings for the snapping.
142 snapControls
143 {
144     //- Number of patch smoothing iterations before finding correspondence
145     // to surface
146     nSmoothPatch 3;
147
148     //- Relative distance for points to be attracted by surface feature point
149     // or edge. True distance is this factor times local
150     // maximum edge length.
151     tolerance 1.0;
152
153     //- Number of mesh displacement relaxation iterations.
154     nSolveIter 30;
155
156     //- Maximum number of snapping relaxation iterations. Should stop
157     // before upon reaching a correct mesh.
158     nRelaxIter 5;
```

```
159 // Feature snapping
160
161 // - Number of feature edge snapping iterations.
162 // Leave out altogether to disable.
163 nFeatureSnapIter 10;
164
165 // - Detect (geometric only) features by sampling the surface
166 // (default=false).
167 implicitFeatureSnap false;
168
169 // - Use castellatedMeshControls::features (default = true)
170 explicitFeatureSnap true;
171
172 // - Detect points on multiple surfaces (only for explicitFeatureSnap)
173 multiRegionFeatureSnap false;
174 }
```

Depois do *snap*, temos o *addLayers*.

O *addLayers* controla a adição de camadas de células próximas às superfícies da geometria.

Essas camadas de células adicionais são frequentemente usadas para melhorar a resolução próxima à superfície e garantir que os efeitos do fluxo ou outras interações físicas sejam capturados com precisão.

O processo geralmente envolve uma série de iterações para garantir que a malha seja refinada adequadamente e atenda aos critérios de qualidade especificados.

Parâmetros do *addLayers*.

```
175 // Settings for the layer addition.
176 addLayersControls
177 {
178     // Are the thickness parameters below relative to the undistorted
179     // size of the refined cell outside layer (true) or absolute sizes (false).
180     relativeSizes true;
181
182     // Per final patch (so not geometry!) the layer information
183     layers
184     {
185         "cylinder"
186         {
187             nSurfaceLayers 4;
188         }
189     }
190
191     // Expansion factor for layer mesh
192     expansionRatio 1.2;
```

```
193 // Wanted thickness of final added cell layer. If multiple layers
194 // is the thickness of the layer furthest away from the wall.
195 // Relative to undistorted size of cell outside layer.
196 // See relativeSizes parameter.
197 finalLayerThickness 0.6;
198
199 // Minimum thickness of cell layer. If for any reason layer
200 // cannot be above minThickness do not add layer.
201 // Relative to undistorted size of cell outside layer.
202 minThickness 0.1;
203
204 // If points get not extruded do nGrow layers of connected faces that are
205 // also not grown. This helps convergence of the layer addition process
206 // close to features.
207 // Note: changed(corrected) w.r.t 17x! (didn't do anything in 17x)
208 nGrow 0;
```

```
209 // Advanced settings
210
211 // When not to extrude surface. 0 is flat surface, 90 is when two faces
212 // are perpendicular
213 featureAngle 60;
214
215 // At non-patched sides allow mesh to slip if extrusion direction makes
216 // angle larger than slipFeatureAngle.
217 slipFeatureAngle 30;
218
219 // Maximum number of snapping relaxation iterations. Should stop
220 // before upon reaching a correct mesh.
221 nRelaxIter 3;
222
223 // Number of smoothing iterations of surface normals
224 nSmoothSurfaceNormals 1;
225
226 // Number of smoothing iterations of interior mesh movement direction
227 nSmoothNormals 3;
228
229 // Smooth layer thickness over surface patches
230 nSmoothThickness 10;
```

```
231 // Stop layer growth on highly warped cells
232 maxFaceThicknessRatio 0.5;
233
234 // Reduce layer growth where ratio thickness to medial
235 // distance is large
236 maxThicknessToMedialRatio 0.3;
237
238 // Angle used to pick up medial axis points
239 // Note: changed(corrected) w.r.t 17x! 90 degrees corresponds to 130 in 17x.
240 minMedialAxisAngle 90;
241
242 // Create buffer region for new layer terminations
243 nBufferCellsNoExtrude 0;
244
245 // Overall max number of layer addition iterations. The mesher will exit
246 // if it reaches this number of iterations; possibly with an illegal
247 // mesh.
248 nLayerIter 50;
249 }
```

Por fim, podemos definir os parâmetros de qualidade que queremos na malha. Isso é feito com o dicionário *meshQualityDict*.

```
250 // Generic mesh quality settings. At any undoable phase these determine
251 // where to undo.
252 meshQualityControls
253 {
254     #include "meshQualityDict"
255 }
```

Para ver e modificar os parâmetros, dê o comando

\$ foamGet meshQualityDict

Esse arquivo vai aparecer na pasta *system*. Aí é só personalizar os parâmetros de qualidade.

Pronto! Rode o comando

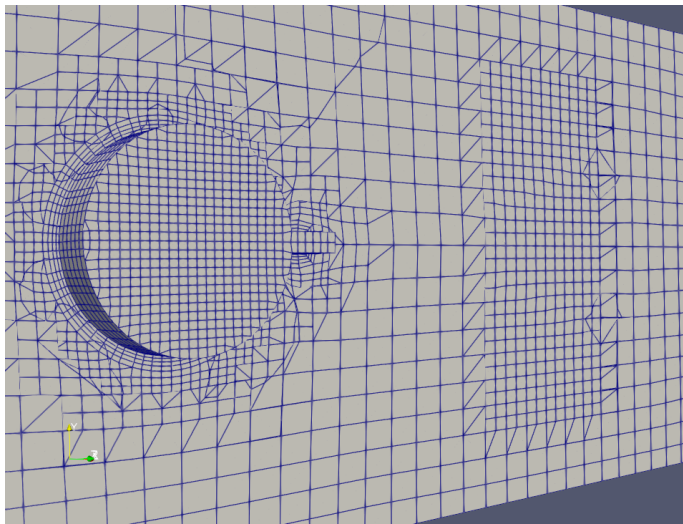
```
$ snappyHexMesh
```

Vão aparecer 3 pastas novas. Cada pasta contém a malha resultante dos processos de *castellate*, *snap* e *addLayers*.

É interessante ver as 3 etapas quando estamos criando a malha e definindo os parâmetros. Depois, quando já for pra valer, o melhor é ter apenas a malha final. Para isso, dê o comando

```
$ snappyHexMesh -overwrite
```

Os detalhes da malha vão ser adicionados em *constant/polyMesh* e em 0.



Resumo de passos e comandos:

- comando *blockMesh*
- abrir o arquivo *cylinder.geo* e salvar *cylinder.stl*
- comando *surfaceFeatures*
- comando *snappyHexMesh -overwrite*

Concluindo.

Modifique os parâmetros e gere outras malhas. Estude a influência de cada parâmetro na malha final.

Crie outras geometrias e gere suas respectivas malhas. Rode alguns casos completos.

O *snappyHexMesh* é um gerador de malhas poderoso. Vale a pena investir um tempo nele se o seu projeto envolve geometrias complexas.

Leia os slides da *Wolf Dynamics* sobre o *snappy*.